Universidad Tecnológica Nacional
Facultad Regional Córdoba
Ing. en Sistemas de Información

EXAMEN PARCIAL 4

Algoritmos y Estructuras de Datos

Fecha: 18/10/2025

Turno T1 - Enunciado:

La aplicación *Poke Genie* permite a los entrenadores coordinar incursiones remotas en distintos gimnasios Pokémon de todo el mundo. Para mejorar la planificación de estos eventos, se necesita desarrollar un programa que permita registrar, analizar y filtrar ***incursiones*** según su nivel y tipo. De cada ***Incursion*** se conoce: Nombre del Pokémon (cadena de caracteres), color del gimnasio (entero entre 1 y 3: 1-Valor (rojo), 2-Sabiduría/Místico (azul), y 3-Instinto (amarillo)), el nivel (entero entre 1 y 5), tipo de incursión (entero entre 1 y 3, donde: 1–Mega, 2–Normal, 3–Dinamax), forma del Pokémon (1 si el Pokémon es *Normal*, 2 si es *Oscuro*), cantidad de jugadores remotos necesarios (entero entre 5 y 10), y cantidad de jugadores adicionales presentes en el gimnasio (entero positivo).

En base a lo anterior, se provee **ya desarrollado**:

- ✓ Un módulo *clase.py* que contiene la definición de la clase **Incursion** con el *método constructor* y el método *__str__()* ya incluidos. NO CAMBIE NADA. NO AGREGUE NADA. DEJE LA CLASE TAL CUAL ESTÁ.
- ✓ Un segundo módulo *principal.py* que contiene una función *main()* con el esquema del menú de opciones ya desarrollado y funcionando, de forma que esa función *main()* sea usada como la función de entrada del programa completo, y una función de validación de carga. En el mismo módulo, se provee ya desarrollada una función *generar_aleatorio()* que crea con valores aleatorios un objeto de la clase **Incursion** y lo retorna ya listo para usar. Los procesos para cumplir con cada opción del menú, son justamente la tarea que debe programar cada estudiante. Las opciones a programar son:

---

1. Cargar un arreglo de registros con los datos de *n* incursiones (cargar *n* por teclado y validar que sea positivo), de manera que en todo momento el arreglo se mantenga ordenado de menor a mayor, por **nombre** del Pokémon. Para esto debe utilizar el algoritmo de inserción ordenada con búsqueda binaria (se considerará directamente incorrecta la solución basada en cargar el arreglo completo y ordenarlo al final, o aplicar el algoritmo de inserción ordenada, pero con búsqueda secuencial). La carga de las incursiones debe ser automática y con base aleatoria (use la función *generar_aleatorio()* que ya tiene disponible en el módulo *principal.py* para crear los objetos). Cada vez que se ingrese en esta opción, el arreglo debe ser creado desde cero y todo contenido anterior debe ser eliminado.

   Ajuste de runner: al ejecutar esta opción, solo cargue por teclado el valor de n, y no muestre nada al terminar de cargar el vector. Solo regrese al menú.

---

2. Mostrar todas las incursiones del arreglo, a razón de un registro por línea. Al final del listado, agregar una línea adicional que muestre el nivel promedio de las incursiones de un tipo t ingresado por teclado.

   Ajuste de runner: al ejecutar esta opción, cargue por teclado el valor de t, muestre el listado pedido usando el método __str__() que ya tiene en la clase, muestre al final el promedio pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

---

3. Contar la cantidad de incursiones para cada combinación posible entre la cantidad de jugadores remotos necesarios y el nivel de incursión (o sea, 6 x 5 contadores en una matriz de conteo). Mostrar todos los contadores que sean mayores a cero.

   Ajuste de runner: al ejecutar esta opción, asegúrese de que la matriz sea de orden 6 x 5 (Y NO 5 x 6). Para mostrarla, recórrala por filas, muestre cada contador pedido indicando a cuántos jugadores y a qué nivel corresponde cada uno (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

---

4. A partir del arreglo, generar un archivo binario que contenga los datos de todas las incursiones que sean de nivel 5 y tipo Mega.

   Ajuste de runner: al ejecutar esta opción, solo genere el archivo pedido y regrese al menú (no muestre nada en la pantalla al terminar).

5. Mostrar el contenido del archivo binario, a razón de un registro por línea, indicando además al final una línea adicional con el porcentaje de incursiones de tipo Dinamax respecto del total de incursiones cargadas en el archivo.

   Ajuste de runner: al ejecutar esta opción, muestre el listado pedido usando el método __str__() que tiene disponible en la clase, muestre al final el porcentaje pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

Criterios generales de evaluación.

- Estructura completa del programa terminado **[máximo: 4 puntos por convenciones de estilo, la estructura de las funciones que desarrolle, el control de ejecución del módulo principal, todo otro elemento que aporte a la estructura del programa]**
- Desarrollo correcto del ítem 1: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 2: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 3: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 4: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 5: **[máximo: 4 puntos]**
- Para aprobar el parcial (con nota mínima de 4(cuatro), el alumno debe llegar a un *porcentaje de al menos 55% del puntaje máximo (eso es, un poco más de 13 puntos)*.